COP 3538 Assignment 1: Student Grade Management System

Grader Contact Information

For any questions or concerns related to assignment grading, please reach out to the grader listed below. All communication must be through the Canvas inbox only.

🎯 **Contact the grader first for any grading issues. If the matter is unresolved, you may then reach out to the instructor.**

Field	Details
Name	Parshatd Govindasamy
Email	pgovi004@fiu.edu

Overview

This **warm-up assignment** reviews fundamental Python programming based on the COP 3410 / COP 3045 prerequisites and object-oriented design and **does not test any specific data structure discussed in this course**. You will implement a `GradeBook` class that stores students and their course grades and answers questions about them (GPA, class averages, and honor roll).

If your implementation passes all autograder tests, you are guaranteed to receive full credit for the automated portion (80 points).

Your Task

Implement every method of the `GradeBook` class in `gradebook.py` per the docstrings in `gradebook_interface.py`. Read that file thoroughly before starting — it is the authoritative specification for every method's exact behavior, including edge cases.

Do not modify `gradebook_interface.py`. It defines the abstract interface your class must implement.

The 6 Required Methods

#	Method	Points
1	`add_student(student_id, name, major)`	13
2	`add_grade(student_id, course_code, score)`	13
3	`remove_student(student_id)`	13
4	`calculate_gpa(student_id)`	13
5	`get_class_average(course_code)`	14
6	`get_honor_roll(min_gpa=3.5)`	14

Each method is graded independently against several test cases (see `gradebook_interface.py` for full behavior specs, including validation rules and boundary conditions). Partial credit is awarded per method based on how many of its test cases pass.

Important dependency note: methods 3–6 require students and grades to already exist in the gradebook. The autograder sets up this starting state by calling **your own** `add_student` / `add_grade` methods. This means if your `add_student` is broken, tests for other methods that depend on it will also fail — implement and test `add_student` and `add_grade` first.

GPA Scale

Your `calculate_gpa()` method must convert each course score to grade points using these exact cutoffs:

Score range	Grade points
90.0 – 100.0	4.0
80.0 – 89.99	3.0
70.0 – 79.99	2.0
60.0 – 69.99	1.0
0.0 – 59.99	0.0

GPA is the average of grade points across all recorded courses, rounded to 2 decimal places.

Environment

- **Python 3.13+ required.** The autograder checks the Python version and will refuse to run on an older version.
- No third-party packages — standard library only.
- **This requirement is mandatory.** Your implementation will be tested using the same autograder framework provided to you by the instructor. This ensures that your code works in the required course environment and prevents compatibility issues where the code works on one computer but not another.

Testing and Verifying Your Submission

You are given **two** tools. They check different things, and you should use both before you submit — **in this order.**

Step 1 — While you develop: `A1_autograder.py` (checks that your CODE is correct)

Files that must be in the same folder to run this:

- `A1_autograder.py`
- `gradebook.py` (your own implementation)
- `gradebook_interface.py`

```
python3 A1_autograder.py
```

It prints a per-method PASS/PARTIAL/FAIL breakdown with the specific case and expected-vs-actual value for anything that fails, followed by your final score.

Where do the test students (like "Alice" or "s1") come from? Nowhere external — there's no data file or database anywhere in this assignment. The autograder simply creates a fresh instance of *your own* `GradeBook` class and calls *your own* methods directly — e.g. it calls `add_student("s1", "Alice", "CS")` on your class the same way you would, then checks what your method returns or does. Every "student" you see mentioned in the test output is just a set of arguments the autograder passes into your code for one specific test, and it's gone again as soon as that test finishes. If your `add_student()` and `add_grade()` are implemented correctly, these test values will work exactly like any other input would.

Use this constantly while you develop — every time you finish a method, run it and see what's still failing.

Step 2 — Right before you submit: `A1_batch_grader.py` (checks that your ZIP is packaged correctly)

`A1_batch_grader.py` **does not grade your code itself.** It runs `A1_autograder.py` against whatever it finds inside your ZIP and reports that result — the score you see in `GRADING_RESULTS/` is the exact same score `A1_autograder.py` would give you directly. The only thing `A1_batch_grader.py` adds is the ZIP-extraction and file-structure check *before* handing off to the autograder.

The autograder above only checks your code. It does **not** check whether your submission ZIP is put together correctly — and a wrongly-packaged ZIP can score a 0 even when your code is perfect, because it never gets graded at all (missing file, wrong filename, extra files, broken ZIP, etc.).

`A1_batch_grader.py` runs the *exact* process your real submission goes through: it extracts a ZIP, checks it contains exactly the required files, and only then runs the autograder on what it finds.

Files that must be in the same folder to run this (call it your verification folder):

- `A1_batch_grader.py`
- `A1_autograder.py`
- `gradebook_interface.py`
- your submission ZIP (e.g. `A1_Yourname.zip`)

⚠️ **Do NOT put your own `gradebook.py` in this folder** — the tool pulls the code to test from inside the ZIP, and deletes any `gradebook.py` here as cleanup once it's done.

Then, from inside that folder, run:

```
python3 A1_batch_grader.py .
```

Read the report written into the new `GRADING_RESULTS/` folder it creates.

Note: this tool is normally used by the instructor to grade a whole class at once, so a couple of its messages are written with that in mind — you'll see `Found 1 submission(s)` and a statistics block showing "Average / Highest / Lowest," which with only your own ZIP in the folder just repeats your own score three times. That's expected, not a bug — ignore the aggregate stats and read your own result.

What this catches that the autograder alone won't: a missing or misnamed file, files nested in an unexpected folder inside the ZIP, a ZIP that won't extract cleanly, or an incomplete README — all things that cost you points for reasons that have nothing to do with whether your code works.

Project Framework

This is the **instructor-provided assignment framework** for this assignment.

```
A1_PROVIDED_FILES/
├── A1_Specification.md          # This document
├── gradebook_interface.py       # Abstract interface (DO NOT MODIFY)
├── skeleton_gradebook.py        # Rename to gradebook.py and implement
├── A1_autograder.py             # Checks your code's correctness
├── A1_batch_grader.py           # Checks your submission ZIP is packaged correctly
└── README.txt                   # Fill in and submit with your code in the ZIP file
```

Submission Policy

- Working in a team is optional — **you may work individually**.
- Each group may include up to **two students**. Exceeding this limit will result in a zero grade for the entire team.
- Select team members **only from within your own section**, not from another cross-listed section, as stated in the syllabus. For example:
  - If enrolled in **COP3538-RVC**, do not form a team with students from **COP3538-RVD**.
  - If enrolled in **COP3538-U01**, do not form a team with students from **COP3538-U02**.
  - If enrolled in **COP3538-UHA**, do not form a team with students from **COP3538-UHB**.
- Submissions violating this requirement will not be graded.
- Team members may vary across assignments.
- Only **one team member** should submit. Include a `README.txt` with accurate team member details.
- If multiple team members submit individually, the assignment will be considered void.
- Unlimited submissions are permitted before the due date without penalty.
- A 10% penalty per day applies from the Due Date until the Until Date. No submissions after the Until Date.

⚠️ **Submit a single ZIP file containing exactly both required files:**

```
# Your implementation
gradebook.py

# Completed – see template
README.txt
```

⚠️ **No other files. Having additional files in the submission ZIP may prevent the batch grader from grading your submission.** The filename format is `A1_Firstname_Lastname.zip` (use the submitting member's name for team submissions).

AI Usage

AI tool use is **permitted and encouraged** within the boundaries of the University's Academic Integrity Policy. Appropriate use of AI will not reduce your grade. You are responsible for ensuring your submission complies with that policy and that you fully understand and can explain any code you submit.

Your `README.txt` must include:

- **SECTION 1: TEAM DETAILS**

- **SECTION 2: AI DISCLOSURE** — state whether you used an AI tool, and if so, the tool name, purpose, prompt(s) used, and what you accepted/modified/rejected. Write "N/A" for each field if you did not use one.
- **SECTION 3: AI CRITIQUE** — have an AI tool critique your `add_grade()` implementation, specifically whether it correctly rejects every out-of-range or malformed input (score outside 0–100, empty course_code, nonexistent student). This is the most bug-prone validation logic in the assignment: a scheduler that silently accepts one bad case will still look correct on typical inputs and only fail on the edge case that triggers it. Paste the AI's response, cite the specific line number(s) in your `gradebook.py` it refers to, state your position (Agree / Disagree / Partially Agree), and explain your position in 2–3 sentences.

Grading Breakdown

Component	Points
Autograder (6 methods)	80
AI Disclosure	2
AI Critique	8
Total	**90** → scaled to your course's PA weighting

Manual component (10 points): the AI Disclosure and AI Critique sections of your `README.txt`, graded by the instructor/TA based on what you actually wrote — see "AI Usage" above for what's expected in each section.
